hashlock.

# Security Audit

## Nodo (dApp)

June 2025 info@hashlock.com.au Hashlock Pty Ltd

# Table of Contents

## CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

# Executive Summary

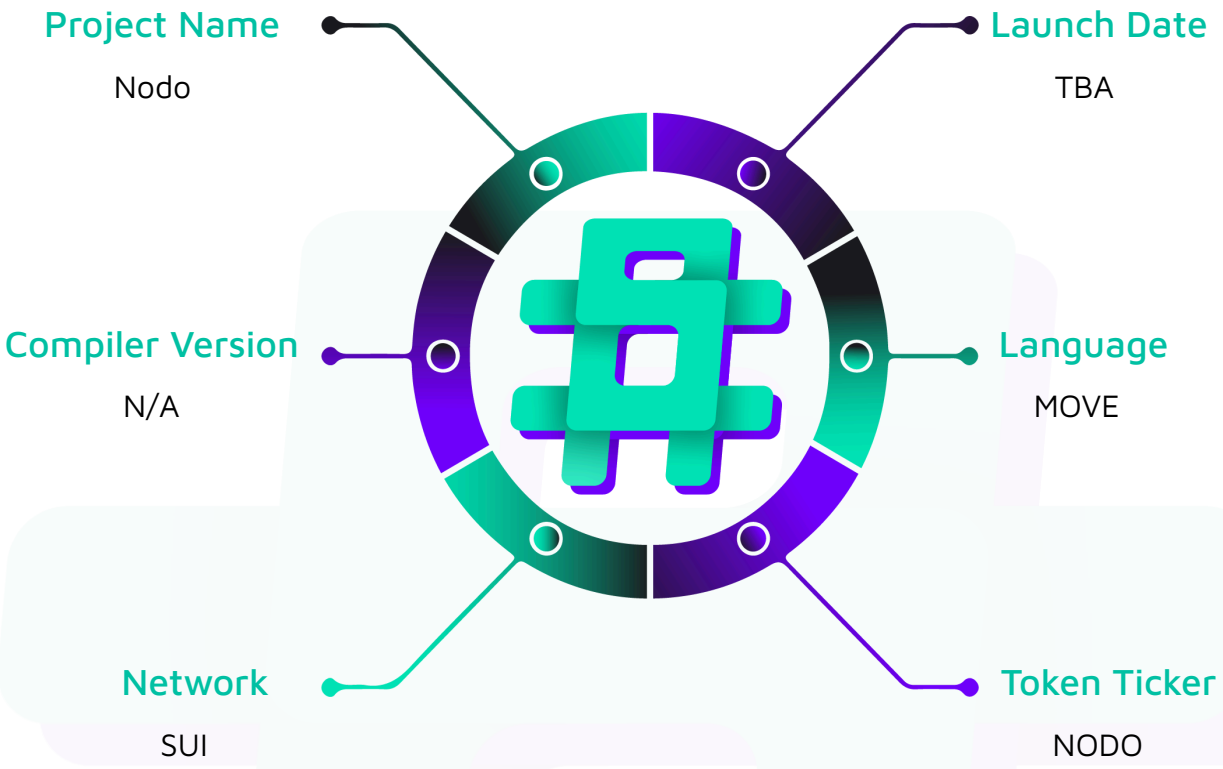
The Nodo team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

# Project Context

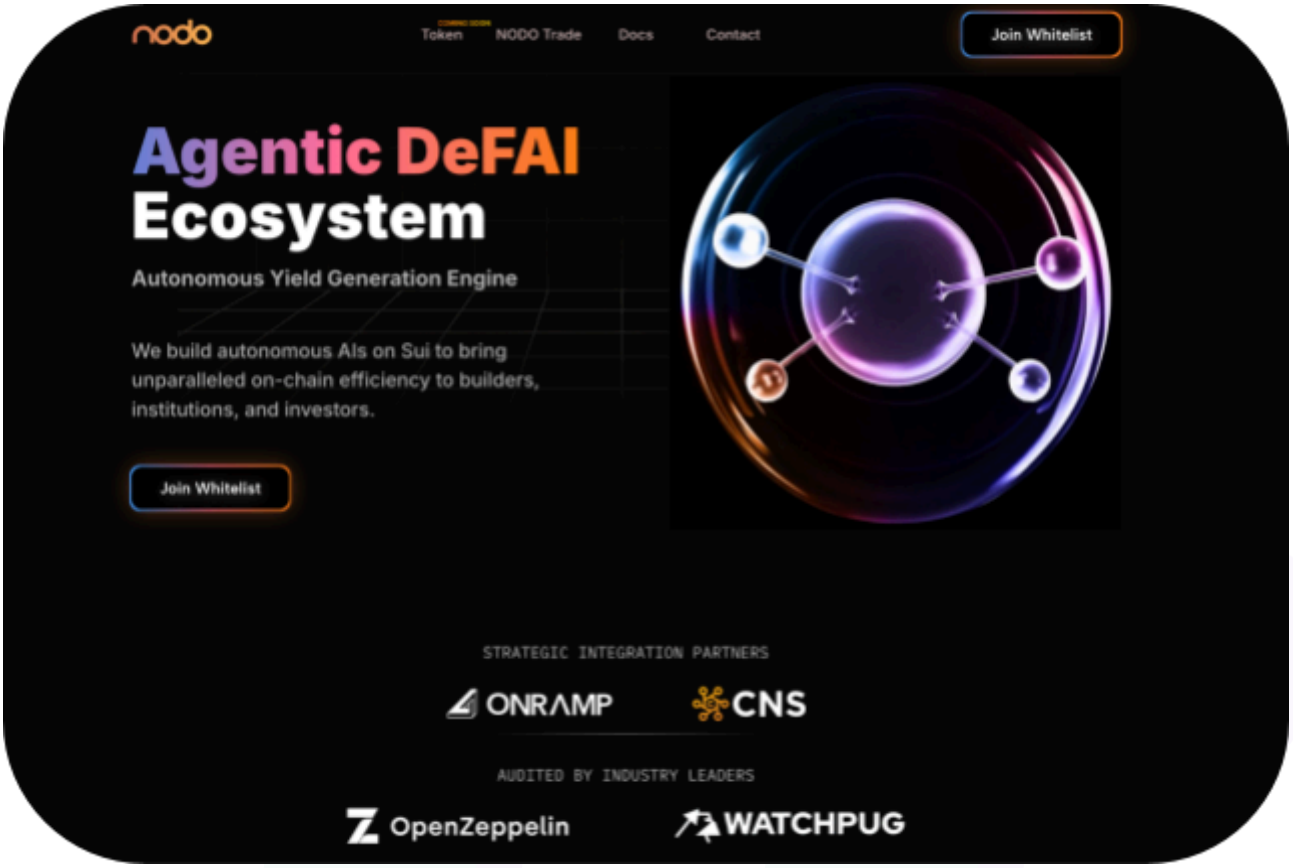
NODO is the multichain AI-powered liquidity management system that powers real-time financial insights & optimizes onchain portfolio management. NODO has built fully on-chain self-governing AI agents based on real-time market conditions to ensure capital efficiency, minimize downside risks, and enable reliable yield compounding for both DeFi users & projects.

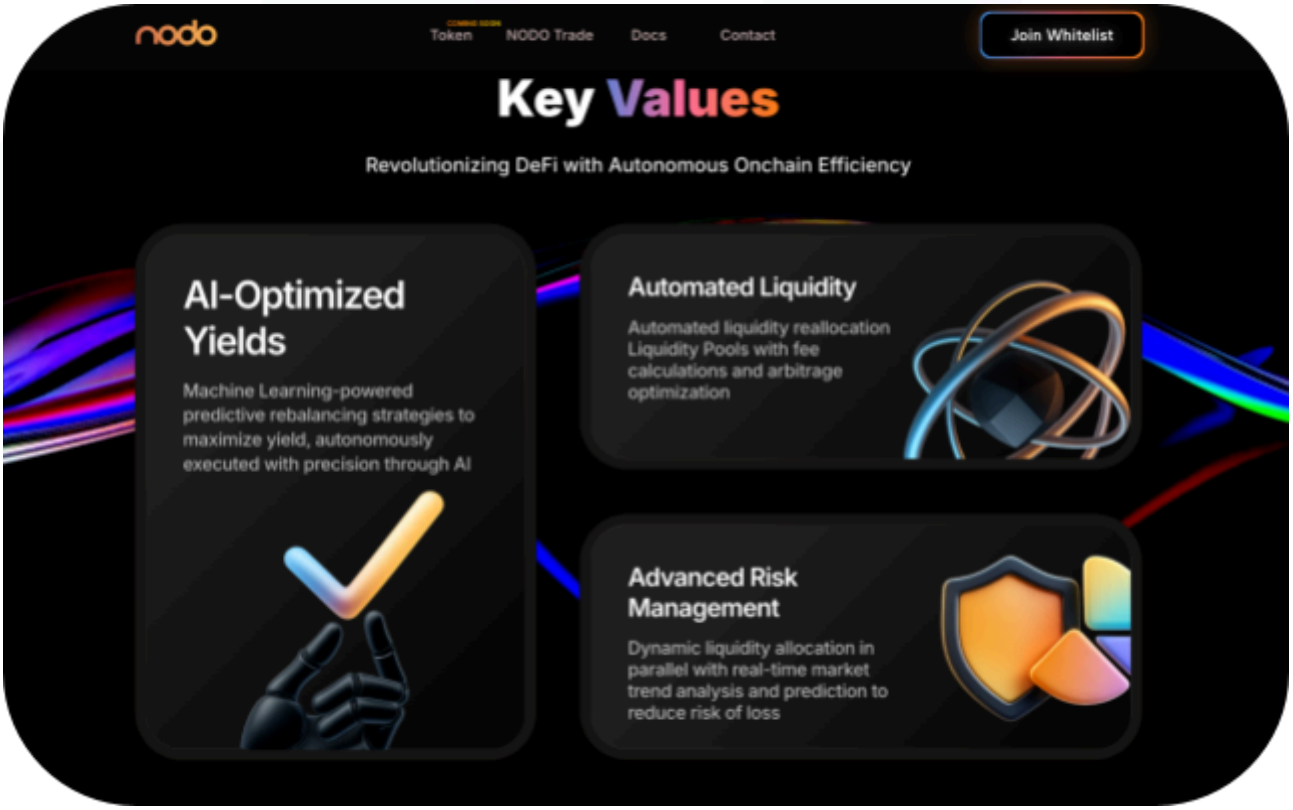
**Project Name**: Nodo
**Project Type**: dApp
**Website**: https://nodo.xyz/
**Logo**:

**Visualised Context:**



**Project Visuals:**



# Audit Scope

We at Hashlock audited the Move code within the Nodo project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

| Description | Nodo Smart Contracts |
|---|---|
| Platform | Ethereum / Move |
| Audit Date | June, 2025 |
| Contract 1 | vault.move |
| Contract 1 MD5 Hash | a22459c927069ee53d8ad46c963c0ff4 |

# Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.


*The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.*

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved.

**Hashlock found:**

3 Medium severity vulnerabilities

3 Low severity vulnerabilities

**Caution:** *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

# Intended Smart Contract Functions

| Claimed Behaviour | Actual Behaviour |
|---|---|
| **vault.move**<br>- Allows users to:<br>  - Supply tokens for loan liquidity<br>  - Take out debt to borrow tokens against their collateral<br>  - Repay debts fully or partially<br>  - Withdraw their initial collateral<br>  - Add STokens<br>  - Propose a liquidationCall<br>  - Claim rewards for protocol interaction<br>- Allows admins to:<br>  - Withdraw revenue earned by the protocol<br>  - Set set the maximum amount to borrow | **Contract achieves this functionality.** |

# Code Quality

This audit scope involves the smart contracts of the Nodo project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

# Audit Resources

We were given the Nodo project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

# Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

# Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

| Significance | Description |
|---|---|
| **High** | High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community. |
| **Medium** | Medium-level difficulties should be solved before deployment, but won't result in loss of funds. |
| **Low** | Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future. |
| **Gas** | Gas Optimisations, issues, and inefficiencies. |
| **QA** | Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code. |

# Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

| Significance | Description |
|---|---|
| **Resolved** | The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue. |
| **Acknowledged** | The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception. |
| **Unresolved** | The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed. |

# Audit Findings

## Medium

### [M-01] total_liquidity - not updated on withdrawals

#### Description

The `total_liquidity` field in the Vault struct is only updated during deposits but never decremented during withdrawals, causing permanent accounting inconsistencies.

#### Vulnerability Details

The deposit() function correctly updates `total_liquidity` :

```
vault.total_liquidity = vault.total_liquidity + (net_amount as u128);
```

However, the `withdraw()` and `redeem()` functions never decrement this value, leaving it inflated after any withdrawal activity. This creates a permanent divergence between the tracked `total_liquidity` and the actual liquidity state.

#### Proof of Concept

```
#[test]

public fun total_liquidity_bug_proof() {

    // User deposits 1000 USDC

    let after_deposit = vault::get_total_liquidity(&vault);

    assert_eq(after_deposit, 1000); // Correctly updated

    // User withdraws everything

    vault::withdraw<USDC, VAULT_TEST>(&mut config, &mut vault, lp_token, &clock,
scenario.ctx());

    vault::redeem<USDC, VAULT_TEST>(&mut config, &mut vault, &clock, scenario.ctx());
```

```
    // BUG: total_liquidity should be 0 but it's still 1000
     let after_withdraw = vault::get_total_liquidity(&vault);
     assert_eq(after_withdraw, 1000); // Never decremented
}
```

**Impact**

Any future features that rely on `total_liquidity` for calculations (fees, rates, reporting) will use incorrect inflated values, potentially leading to wrong financial calculations or misleading vault metrics.

**Recommendation**

Update `withdraw()` function to decrement `total_liquidity`:

```
vault.total_liquidity = vault.total_liquidity - (amount as u128);
```

**Status**

Resolved

## [M-02] Management fee - total calculation uses wrong fee struct

### Description

The `withdraw_management_fee()` function incorrectly uses `vault.performance_fee.total_fee` instead of `vault.management_fee.total_fee` when calculating the new management fee total.

### Proof of Concept

```
vault.management_fee.total_fee = vault.performance_fee.total_fee + amount;
```

### Impact

This causes incorrect management fee accounting

### Recommendation

Fix the calculation to use the correct fee struct:

```
vault.management_fee.total_fee = vault.management_fee.total_fee + amount;
```

### Status

Resolved

## [M-03] Duplicate LP tokens - can be added causing position inflation

### Description

The `add_lp()` function allows the same LP token ID to be added multiple times to the vault, creating duplicate position entries that can be withdrawn multiple times.

### Vulnerability Details

The `add_lp()` function lacks duplicate checking and simply adds any LP ID to the vector:

```
vector::push_back(vec, lp_id); // No duplicate check
```

This allows the same LP token to be added multiple times

### Proof of Concept

```
#[test]
public fun duplicate_lp_bug() {
    // Add same LP 3 times
    vault::add_lp(&mut config, &mut vault, amm, lp_id, &access_cap);
    vault::add_lp(&mut config, &mut vault, amm, lp_id, &access_cap);
    vault::add_lp(&mut config, &mut vault, amm, lp_id, &access_cap);

    // Proof: 3 positions but all same ID
    let lps = vault::get_lp_ids_by_amm<USDC, VAULT_TEST>(&vault, amm);
    assert_eq(vector::length(&lps), 3);
    assert_eq(*vector::borrow(&lps, 0), lp_id);
    assert_eq(*vector::borrow(&lps, 1), lp_id);
    assert_eq(*vector::borrow(&lps, 2), lp_id);

    // Withdraw same LP 3 times
    let w1 = vault::withdraw_lp(&config, &mut vault, amm, &access_cap);
    let w2 = vault::withdraw_lp(&config, &mut vault, amm, &access_cap);
    let w3 = vault::withdraw_lp(&config, &mut vault, amm, &access_cap);

    assert_eq(w1, lp_id);
    assert_eq(w2, lp_id);
```

```
    assert_eq(w3, lp_id);
}
```

**Impact**

- Multiple withdrawal claims on the same LP token
- Position count inflation causing incorrect vault metrics
- Accounting inconsistencies in LP position tracking

**Recommendation**

Add duplicate checking in `add_lp()` to prevent the same LP ID from being added multiple times:

```
let vec = vec_map::get_mut(&mut vault.lp_storage, &amm);

if (!vector::contains(vec, &lp_id)) {
vector::push_back(vec, lp_id);
}
```

**Status**

Resolved

# Low

## [L-01] Unused owner field in Vault struct

### Description

The owner field in the Vault struct is set during initialization but never used throughout the contract, representing dead code.

### Vulnerability Details

The owner field is initialized in `init_vault()`:

```
owner: ctx.sender(),
```

### Impact

Dead code that may confuse developers

potential misleading assumption that `owner-based` access control exists

### Recommendation

Either remove the unused owner field entirely to reduce storage costs, or implement `owner-based` functionality

### Status

Resolved

## [L-02] Incorrect error code for asset existence check

### Description

The `spend_harvest_asset()` function uses an incorrect error code when checking if a harvest asset exists in the vault.

### Vulnerability Details

When checking if a harvest asset exists, the function uses `ENotEnoughBalance`, This error code is misleading since the real issue is that the asset type doesn't exist in the vault, not that there's insufficient balance.

### Proof of Concept

```
assert!(bag::contains(&vault.harvest_assets, key), ENotEnoughBalance);
```

### Impact

Users and integrators receive confusing error information

### Recommendation

Use a more appropriate error code such as `EAssetNotFound` or `EHarvestAssetNotExist` to clearly indicate that the requested asset type doesn't exist in the vault's harvest assets.

### Status

Resolved

## [L-03] Missing cleanup logic for empty harvest assets

### Description

The `spend_harvest_asset()` function lacks cleanup logic to remove empty coin objects and stale keys after assets are fully spent.

### Vulnerability Details

After spending tokens via `split()`, if all tokens are consumed, the empty `Coin<CoinType>` object remains in the bag permanently.

### Proof of Concept

```
#[test]
public fun harvest_keys_not_cleaned_up() {
    // Add 100 TEST tokens
    vault::add_harvest_assets<USDC, VAULT_TEST, TEST>(&mut config, &mut vault, test_coin, &access_cap);

    // After adding: keys should have 1 entry
    let after_add = vault::get_harvest_asset_keys(&vault);
    assert_eq(vector::length(&after_add), 1);

    // Spend ALL tokens (empty the asset completely)
    let spent = vault::spend_harvest_asset<USDC, VAULT_TEST, TEST>(&mut config, &mut vault, 100, &access_cap, scenario.ctx());

    // Balance is now 0 (asset empty)
    let balance = vault::get_harvest_asset_balance<USDC, VAULT_TEST, TEST>(&mut vault);
    assert_eq(balance, 0);
```

```
    // BUG: Keys still show 1 entry even though asset is empty!
    let after_spend = vault::get_harvest_asset_keys(&vault);
    assert_eq(vector::length(&after_spend), 1); // Still 1, should be 0
}
```

**Impact**

- Storage bloat from accumulating empty coin objects over time
- Misleading asset information for external integrations

**Recommendation**

Implement cleanup logic to check if the remaining coin balance is zero after spending, and if so, remove the empty coin from the bag and the corresponding key from the `harvest_asset_keys vector`.

**Status**

Resolved

# Centralisation

The Nodo project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

# Conclusion

After Hashlock's analysis, the Nodo project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

# Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

**Manual Code Review:**

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

**Vulnerability Analysis:**

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

**Documenting Results:**

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

**Suggested Solutions:**

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

# Disclaimers

## Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

## Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

# About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

**Website:** hashlock.com.au
**Contact:** info@hashlock.com.au